

Vst Fingers

An Orchestra in Your Hand

Samuel Nnawuihe Echefu
MUS 207
University of Victoria
Victoria, BC Canada
samuel.n.echefu@gmail.com

Leo Rettie
MUS 207
University of Victoria
Victoria, BC Canada
leojrettie@protonmail.com

CONCEPT

1.1 Overview of Project

Our project creates a connection between the physical world and manipulating the technology in order to output sounds using just finger movement and camera detection.

1.2 Target Users and Their Expected Use Cases

Performers, bands, orchestras, and individual artists would make great use of this project as it is incredibly applicable to any person who has used a DAW or a VSTi. Performers can adjust the Python code to add visual effects to the hands to create emphasis, bands can increase their instrumental capacity by 4 at the minimum, and individual artists can play around for inspiration or improvise composition so they can create beautiful compositions.

CONTEXT

2.1 Technical Context

In order to use this system a user would need all of the following applications downloaded on their operating system: VS Code (or any Code Editor), Max/Msp 9, Python, and a working web cam or camera.

2.1.1 Max 9

Since the Python, VSTi, and server connection happen through Max 9. Max 9 is a necessity to this project and allows for OSC sound support which is vital for Python communication.

2.2 Python Libraries

The necessary Python libraries used in this system are Media Pipe, PythonOsc, and OpenCV.

2.2.1 VST and VSTi

Visual Studio Technology (VST) are virtual instruments that can be communicated with through a digital audio workstation (DAW). A VST comes in many varieties, including a VST instrument (VSTi) and VST effects (VSTFX). In this system, we focus on the use of VSTis and VST synthesizers like Serum, SurgeXT, and Xpand. [1]

2.2.2 Media Pipe

Media Pipe is a Google framework for machine learning perception, more commonly utilized for hand, face, pose, and object detection.[2]

2.2.3 PythonOsc

PythonOsc is an open sound control messaging system for communicating in Max 9 using the User Datagram Protocol (UDP) and Transmission Control Protocol(TCP). [3]

2.2.4 OpenCV

OpenCV is an open-source computer vision library for object detection, image processing, and video analysis. [4]

IMPLEMENTATION

3.1 Description of Tools

Implementing the necessary tools to create this system device consists of intertwining Python and Max 9. Using AI finger detection, input processing, and one-to-many, many-to-many, and many-to-one mappings in Max 9 gives a user control over an extensive library of sounds in various VSTs that a user can set up.

3.2 Python Libraries Used for Max 9

3.2.1 Media Pipe Processing

Media pipe in our project only focused on hand detection specifically, finger detection. The pre-trained AI model detects the hand in real-time and collects the exact points of the hand (fingers, fingertips, knuckles, and thumb). Implement a visual function to check whether the detection captures the correct location of the fingers. This feature is mainly used for testing and should be ignored in the presentation as it can clutter the screen. It also allows the detection to be more reactive but unnecessary.

3.2.2 PythonOsc Communication

PythonOsc was vital in creating a genuinely interactive system. In our project, we programmed Max to open Port 5500, and through

that port, we sent a list of whichever fingers were up, determined by 0 (down) and 1 (up), and processed them through a message and various integer objects, which are sent to decide when to max out the volume of a VST or to mute it.

3.2.3 OpenCV Design

OpenCV implementation for our project was simple used to deploy the computer camera or third-party camera, set the width and height of the display, and create a frame for the screen where object detection works best. On its own, this library is more for aesthetics in our project and doesn't use the full potential of the library, but it is still essential for testing and deploying the camera; without it, we would not be able to create this idea.

3.3 Technical Challenges

This project's technical challenges included controlling which VST to play, depending on which finger. Communicating with Max 9 with Python was already challenging in the development process, as it was difficult to send the correct values without overheating the laptop. Our first challenge was determining whether to send what finger is up individually or send a list of which fingers are up, which allowed us to declutter our Max patch. Another issue was shutting the sound off for each VST, in which we used SurgeXT for testing. We then figured out we must communicate with the exact name for Master Volume that the SurgeXT developing team named the parameter, and then we could turn off the overall sound of a VST or maximize the volume. All in all, issues like adding multiple VSTs were simple copy-and-paste as they are formatted to work in all cases.

STRENGTHS AND WEAKNESSES

4.1 Strengths

Multiple libraries and different software types can come with great strengths and weaknesses. The strengths of a system like this can allow for entire orchestras with only two to three people, considering there would be ten to fifteen fingers and possible preset mappings. The software allows for any VST compatible with Max 9 or downloaded on a user's computer, which opens the range of our system and allows for limitless possibilities for creative endeavours. The project can create beautiful compositions, one-person concerts, and personal projects for beginner composers.

4.2 Overall Weaknesses

A weakness of this project is that the user must physically set each preset to whatever VST they would like to use, which can cause awkward transitions. Another overall weakness in the output is controlling BPM. the system can only play on a rhythm manually set by a metro object. You can play many notes, but the user must manually set them. A final weakness

4.3 Input and Output

The input from the camera can control just the master volume, which uses a simple 0 and 1 input change, making it very reliable, as the Python code continually sends an integer value. In Max 9, after processing the VST output and determining which VST should play (depending on whichever finger is up), audio output will commence.

4.4 Mappings

The mappings for the system are relatively simple, including many-to-many using a kslider object to send midi values to each VST to be played. An unpack one-to-many object to send the list of integers from the Python file, which is consistent and can only fail due to physical tampering with the object.

4.4 Reliability

The system is very consistent, and it can work on multiple different computers and operating systems that have the capabilities to run Max 9 or the necessary VSTs a user would like to implement. Due to the simplicity of the Python to Max communication, it would be difficult for the system to stop functioning. Due to the consistency and simplicity of the system, it is a very reliable system.

4.5 Depth

The system's use of a user's fingers can be very engaging for performances, and it's intertwining between different VSTs that are constantly updating with new libraries and presets. Allowing for constant engagement.

4.5.2 Layers

The system's ability to use any VSTs that a finger could be mapped to allows for infinite sounds and settings, which becomes a gateway for incredible performances.

4.6 Final Thoughts

The strengths and weaknesses of the system can be mitigated with future development and are only weaknesses due to development time and budget concerns. From sections 4.3 - 4.5, the weakness that applies to each can only be from user misuse.

FUTURE IMPROVEMENTS

5.1 The Future

The future capabilities of this system are massive. Earlier stated, beautiful compositions, big orchestras, and personal endeavours. The future of this system is very bright. Increasing the number of hands an individual can use would further break into a new realm of possibilities. Allowing alternate VST effects to filter, reverb, delay, and EQs and creating mixing capabilities of the sound to soften the overall sound.

REFERENCES

- [1] Audiophile ON. *What is VST?: A Guide to Virtual Studio Technology*. Available at: <https://www.audiophile.com/news/vst-guide-to-virtual-studio-technology>
- [2] Google AI. *Hand landmarks detection guide*. Available at: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker (Accessed: 6 April 2025).
- [3] Python-osc. *OSC server and client in pure Python*. Available at: <https://python-osc.readthedocs.io/en/latest/> (Accessed: 6 April 2025).
- [4] OpenCV. *GUI Features in OpenCV*. Available at: https://docs.opencv.org/5.0.0-alpha/dc/d4d/tutorial_py_table_of_contents_gui.html (Accessed: 6 April 2025).